



**NATIONAL INSTITUTE OF BUSINESS MANAGEMENT
SCHOOL OF COMPUTING AND ENGINEERING**

DSE 26.1F

OBJECT ORIENTED PROGRAMMING - COURSEWORK

COURIER MANAGEMENT SYSTEM

GROUP -14

CODSE261F-017

CODSE261F-019

CODSE261F-031

CODSE261F-061

Outline

1. Overview
2. OOP
3. JAVA
4. Decision And Selection
5. Problem Identification And Analyze
6. Code Structure
7. UML Class Diagram
8. ER Diagram
9. UI
10. Code Samples
11. Test And QA

1. Overview

Modern enterprises thrive in the current world due to the effective implementation of logistics and supply chain management tools. The development of e-commerce platforms has significantly improved the flow of physical products across national and international levels.

For modern courier services, the processing of this flow manually or by legacy systems creates numerous operational flaws:

- Manual accounting of the order data
- Disorganized courier process
- Non-automated calculations

Consequently, this results in delayed delivery, unaccounted or lost shipments, incorrect prices of the services, and reduces the trust of the customers.

Taking the above issues into account, this coursework project aims to develop and implement an independent Courier Management System (CMS). Built in the Java language using an object-oriented architecture, the proposed CMS will be a computerized solution intended to automate the administration of order placement, allocation of the delivery, its monitoring, and calculation of the price.

2. OOP - Object Oriented Programming

First of all, it should be noted that the stability of a large-scale system, such as CMS, largely depends on the chosen programming paradigm. In the case of a procedural approach to software development, the model is based on functions. The linear nature of such software makes it difficult to manage when the complexity of the application grows. On the contrary, the Object-Oriented Programming (OOP) paradigm models software design around data structures, called objects, that are the direct representation of the real world. Given the logistics industry with numerous interrelated moving entities, such as customers, consignments, sorting stations, and employees, OOP seems to be a natural choice. Application of the core OOP principles will provide the following benefits for the design of the CMS:

Encapsulation

- Bundles the object's data (fields) and its behavior (methods) in one logical entity.
- Hides internal mechanics from the other elements of the system.
- Defines all important attributes (actual weight, customer contact info, financial characteristics) as private variables.
- Restricts access to attributes exclusively through public getters and setters with strict validation business logic.
- Guarantees data consistency (e.g., throwing exceptions if an employee inputs a negative weight or invalid email).

Inheritance

- Allows developers to create an extended hierarchical structure for a wide variety of cargo.
- Replaces the need to create separate classes for every single parcel variation.
- Establishes a central abstract class named Parcel containing common attributes like ID, destination address, and size.

- Enables specialized subclasses (StandardParcel, ExpressParcel, HeavyParcel) to inherit parent features and add specific traits (insurance fees, handling charges).
- Helps to avoid duplication of the code, to standardize the system, and to ease maintenance.

Polymorphism

- Helps to represent the same interface in multiple forms depending on the concrete type of object.
- Provides a structural contract via the abstract method `calculateShippingFee()` inside the Parcel class.
- Forces every subclass inheriting the base class to have its own unique implementation of the method.
- Executes the appropriate pricing formula dynamically when the central processing engine iterates through a collection.
- Eliminates the use of complicated branching statements such as if-else or switch during processing.

Abstraction

- Implies the hiding of the implementation details and exposure of the key behavior of the objects.
- Uses abstract classes and structural interfaces to create predictable architectural patterns.
- Ensures interaction with the dispatching workflow takes place at a high level rather than dealing with low-level database drivers.
- Minimizes dependence between different elements of code, making scaling, modification, and extension easy tasks.

3. JAVA

Java is a high-level, object-oriented programming language designed for platform independence. Developed by Sun Microsystems (now Oracle) in 1995, it relies on the principle of "Write Once, Run Anywhere" (WORA), meaning compiled Java code runs on any device supporting the Java Virtual Machine (JVM).

A standalone application operates locally on a user's machine without requiring a constant network connection or external web browser infrastructure. In logistics hubs, regional sorting centers, and dispatch kiosks, standalone desktop applications offer distinct advantages: **High Performance & Responsiveness:** Local processing avoids network latency, ensuring immediate responses during high-volume parcel check-ins and barcode scans. **Offline Operational Continuity:** Sorting facilities can continue logging shipments, printing waybills, and managing local records even during WAN connectivity outages. **Direct Hardware Integration:** Java desktop applications easily interface with local hardware peripherals such as receipt printers, digital weight scales, and handheld barcode scanners.

Operational Advantages & Technical Architecture

- **Enhanced Data Security & Privacy:** Keeping operational logs and customer shipping details in local relational databases adds an extra layer of security. By storing data locally instead of sending it over the network, you shield it from online threats, interception, and unauthorized outside access.
- **Granular Desktop Control:** Using Java Swing or JavaFX for the graphical user interface (GUI) makes complex desktop workflows much easier to handle. It allows for smooth multi-window navigation, handy keyboard shortcuts, and custom layout management designed specifically for high-speed data entry by warehouse staff.
- **Predictable Maintenance & Low Resource Overhead:** Running as a self-contained local application removes reliance on third-party web hosting environments, which cuts down server infrastructure costs and simplifies software updates across local dispatch terminals

4. Decision and selection

Architecture & Application Type

Choice Made: Standalone Java desktop application using Java Swing.

- **Why over CLI:** Console applications lack visual feedback, input validation, and smooth multi-window navigation needed by non-technical operators.
- **Why over Web:** Standalone apps remove web hosting reliance, eliminate network latency during data entry, and work offline during internet outages.

Database & Persistence

Choice Made: Local SQLITE database connected via JDBC.

- **Why over Files:** Text files lack transaction safety and risk data corruption during crashes, while SQLITE guarantees ACID compliance across application restarts.
- **Why over NoSQL:** The system relies on structured, relational data (such as Sellers, Packages, and Couriers), which SQLITE protects through foreign keys and Data Access Objects (DAOs).

Tech Stack

- **Java 17 LTS:** Delivers strong performance, cross-platform compatibility, and modern language features like pattern matching and optimized memory handling.

Core Design Choices

- **DAO Pattern:** Separates database logic from the user interface, ensuring database updates won't break UI screens like SellerUI.
- **JUnit 5 Testing:** Automates unit tests for core formulas, weight validation, and state logic to prevent bugs.

5. Problem Identification and Analysis

Real-World Entities

Entity	Domain Role	Attributes	Key Operations
Reserver	receiver of parcels	customerID, name, phone, email, address	seeInfo()
Parcel	Physical package being routed	name, weight, destination	showInfo()
Driver	Dispatch staff executing delivery	driverID, name, phone, email, vehicleType, isAvailable	toggleAvailability() updateStatus()
Session	Binding entity linking customer, parcel, and driver	trackingID, customer, parcel, driver, admin, status	createBill(), calculateCost()
Sender	Sender of the parcels	senderID, name, phone, email, address, age	send()
Admin	How mange and responsible for sessions	adminID, name, phone, email, address, age	register() update() remove() modify()

Functional Requirements

Manage Orders: Allows sellers to easily create, update, and manage delivery orders.

Track Packages: Provides real-time status updates (such as *Pending*, *Dispatched*, or *Delivered*).

Assign Couriers: Matches packages to available couriers based on their schedules and workload.

Calculate Costs & ETAs: Automatically determines delivery fees and estimated arrival times based on package weight and distance.

Generate Reports: Displays helpful statistics to administrators regarding deliveries, active couriers, and revenue summaries.

Non-Functional Requirements

Reliable Storage: Saves all order and user data securely using a relational MySQL database so information persists across application restarts.

Simple Interface: Delivers clean graphical user interfaces (SellerUI and main admin dashboard) that require no special training to navigate.

Domain Selection Justification

Great for Learning OOP: A courier system naturally consists of distinct, interacting parts (Packages, Couriers, Sellers, Reports), making it ideal for practicing core Object-Oriented Programming concepts like Encapsulation, Inheritance, Polymorphism, and Abstraction.

Real-World Practicality: Features like automated fee calculations, state-driven tracking updates, and relational data persistence mirror commercial enterprise logistics software.

Solves a Real Problem: Automating delivery management eliminates human error and streamlines package shipping for growing businesses.

6. Code Structure

System Architecture & OOP Principles Applied The architecture is structured around clean object-oriented separation of concerns [cite: 1]: Abstraction: An abstract base class Parcel defines common attributes (parcel ID, weight, base fee) and abstracts the fee calculation method calculateShippingFee(), which must be implemented by concrete subclasses.

Inheritance: Specialized parcel types—StandardParcel, ExpressParcel, and HeavyParcel—inherit from Parcel, inheriting core properties while extending specific features (e.g., insurance fees, handling surcharges). Polymorphism: The system processes collections of Parcel references dynamically. Calling calculateShippingFee() on any parcel instance executes its specific overridden pricing logic. Encapsulation: All class fields are declared private. Controlled access is provided via getters, setters, and operational methods with strict validation checks (e.g., preventing negative weights).

The **Courier Management System** is built using Java and structured cleanly into distinct packages. This modular layout keeps the user interface, business rules, and database operations separate, making the codebase easier to read, test, and maintain.

Single directory file structure

...

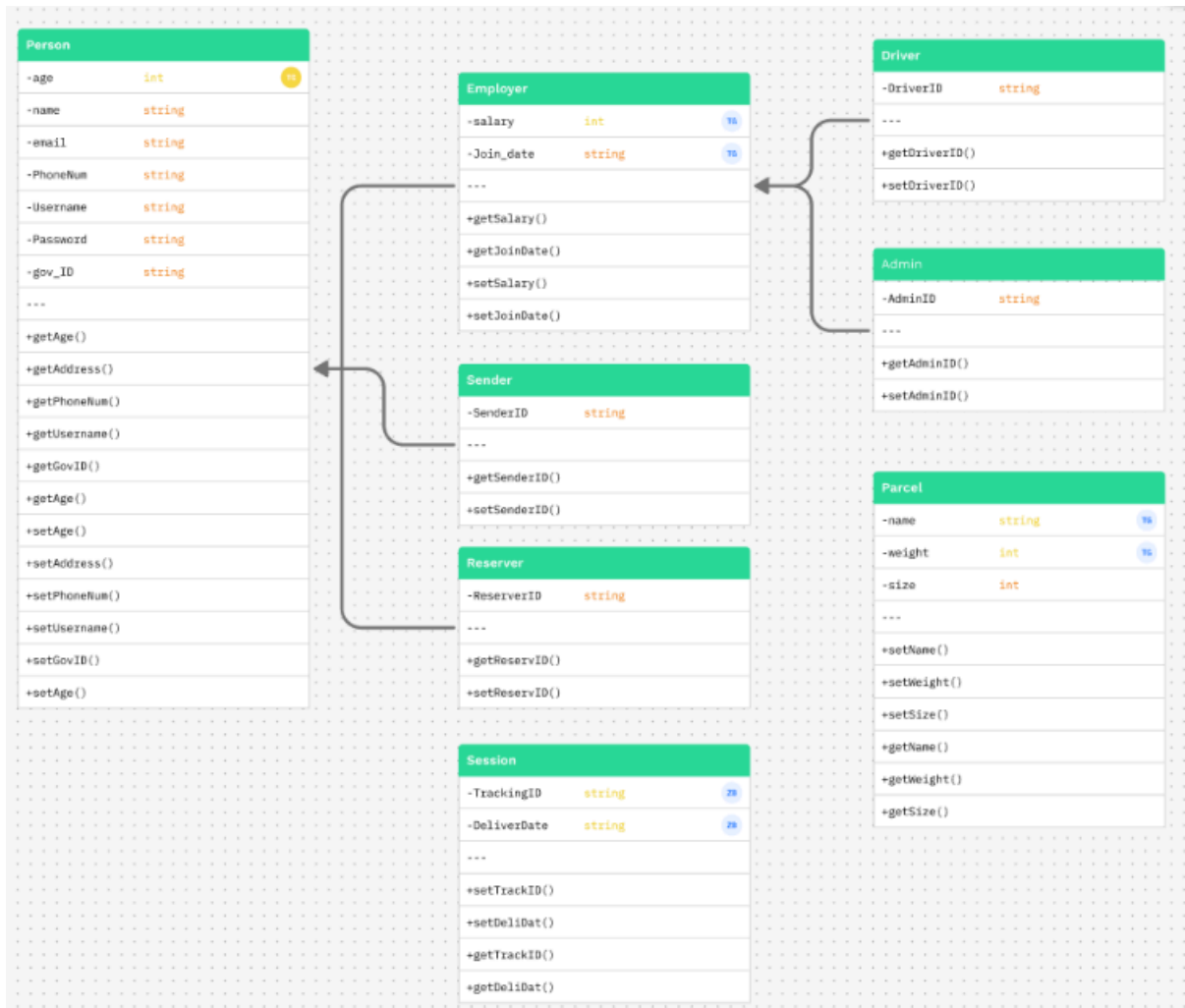
```
App.java      # main entry point
Theme.java    # colours + fonts (orange/yellow theme)
Styler.java   # utility methods that style all Swing elements
CoreUI.java   # base UI panel (frame, back button, centred layout)
-
HomeUI.java   # tracking ID input -> ReserverUI, link -> LoginUI
LoginUI.java  # credentials -> AdminUI / DriverUI / SenderUI / Home
AdminUI.java  # table of all active parcels
DriverUI.java # view + update parcel status
SenderUI.java # send a new parcel (form)
ReserverUI.java # view current state + details of a traced parcel
```

```

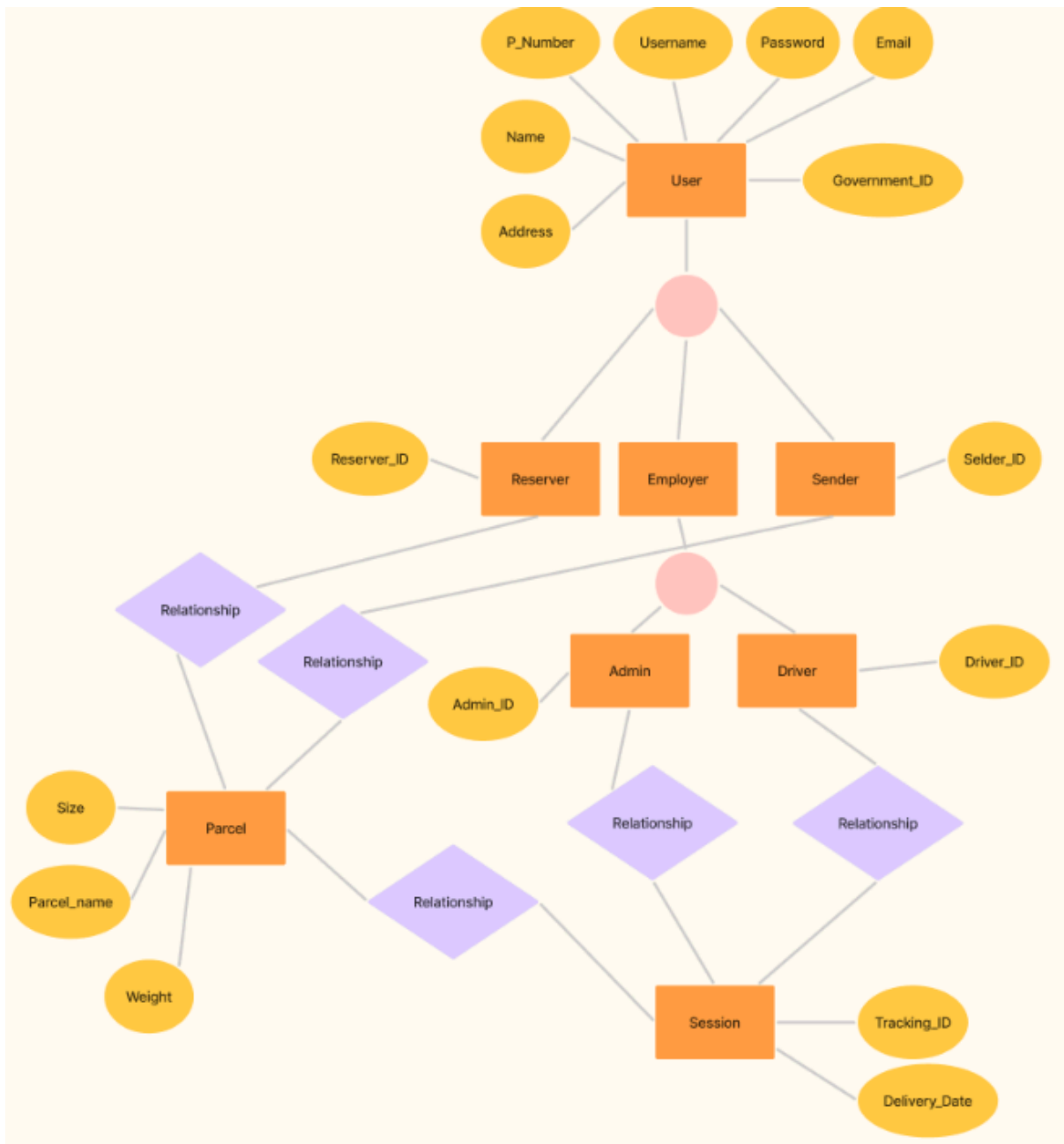
-
User.java      # base user model
Employer.java  # extends User (salary, joiningDate)
Admin.java     # extends Employer
Driver.java    # extends Employer (+ licenseNo)
Sender.java    # extends User
Reserver.java  # extends User
Parcel.java    # parcel model
Session.java   # tracking session model (status + dates)
-
DBmanage.java  # SQLite connection, schema, seed data
CoreDAO.java   # base Data Access Object
AdminDAO.java
DriverDAO.java
SenderDAO.java
ReserverDAO.java
ParcelDAO.java
SessionDAO.java
-
Test.java      # single-file test suite
-
sqlite-jdbc.jar  # SQLite JDBC driver
slf4j-api.jar   # logging API used by the driver
build.ps1       # reproducible script that compiles + builds cms.jar
cms.jar         # final standalone build
README.md
'''

```

7. UML class diagram

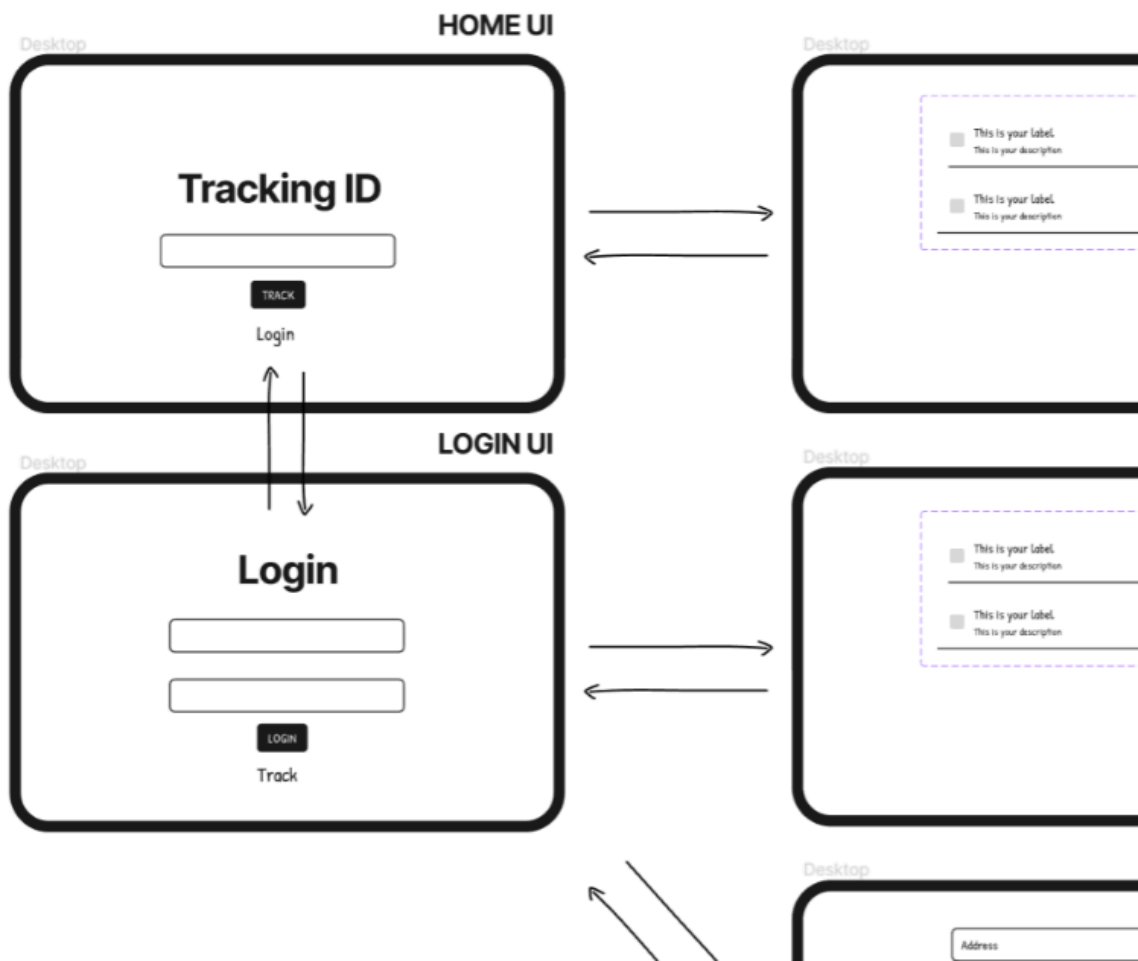


8. ER Diagram



9. UI - User Interface





 Courier Management System

Courier Management System

Courier Management

Track your parcel

Tracking ID

Track Parcel

[Log in here](#)

 Courier Management System

Courier Management System

Parcel Details

Tracking ID: CMS000001

Parcel Laptop Gift

Weight 2.4 kg

Size 40.0 cm

Sender address 7 Market St, Galle

Receiver address 3 Lake Rd, Jaffna

Status Out for Delivery

Delivery by 2026-09-02

[Track Another](#)

[< - Back](#)

10.Code Samples

The main entry point of the application - App.java

```

'''
import javax.swing.*;
import java.awt.*;

/**
 * App - main entry point and single-window controller of the Courier
 * Management System.
 *
 * The whole interface lives inside one {@link JFrame}. The first screen is
 * installed before the window is made visible, and every later navigation
 * installs a fresh content panel and re-validates/repaints the window. One
 * window, one screen at a time, so an empty or stale frame is never shown.
 */
public class App {

    private static JFrame frame;

    private App() {
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater() -> {
            try {
                DBmanage.init();
            } catch (Exception e) {
                JOptionPane.showMessageDialog(null,
                    "Database init failed: " + e.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
            }
            return;
        }
    }
}

```

```

        frame = new JFrame("Courier Management System");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(640, 720);
        frame.setMinimumSize(new Dimension(480, 540));
        frame.setResizable(true);
        frame.setLocationRelativeTo(null);

        // The first screen is placed in the window before it becomes
        // visible, so the window never appears empty.
        install(new HomeUI());

        frame.setVisible(true);
    });
}

/**
 * Shows the given screen, replacing whatever was shown before. A fresh
 * content panel is installed each time and the window is fully re-validated
 * and repainted, so the new screen is always drawn.
 */
public static void open(final CoreUI screen) {
    SwingUtilities.invokeLater() -> {
        install(screen);
        frame.validate();
        frame.repaint();
    });
}

private static void install(final CoreUI screen) {
    JPanel content = new JPanel(new BorderLayout());
    content.add(screen, BorderLayout.CENTER);
    frame.setContentPane(content);
    screen.requestFocusInWindow();
}

```

```

    }
}
'''

```

Class that inherited by all other UI classes - CoreUI.java

```

'''
import javax.swing.*;
import java.awt.*;

/**
 * CoreUI - base class for every screen.
 *
 * A plain JPanel with a simple BorderLayout: a title bar on top, the screen's
 * content in the middle, and a back button at the bottom. Screens are swapped
 * by {@link App} inside a single window.
 */
public abstract class CoreUI extends JPanel {

    protected JPanel body;
    protected JButton backBtn;

    public CoreUI() {
        setLayout(new BorderLayout());
        setBackground(Theme.BG);

        add(titleBar(), BorderLayout.NORTH);

        // Simple centred body: content is placed in the middle and grows to fit.
        body = new JPanel(new BorderLayout());
        body.setBackground(Theme.BG);
        add(body, BorderLayout.CENTER);

        add(footerBar(), BorderLayout.SOUTH);
    }
}
'''

```

```
}
```

```
private JPanel titleBar() {
    JPanel top = new JPanel(new BorderLayout());
    top.setBackground(Theme.YELLOW_SOFT);
    top.setBorder(BorderFactory.createEmptyBorder(12, 16, 12, 16));
    JLabel title = Stylier.h2("Courier Management System");
    title.setForeground(Theme.ORANGE_DARK);
    top.add(title, BorderLayout.WEST);
    return top;
}
```

```
private JPanel footerBar() {
    JPanel footer = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 8));
    footer.setBackground(Theme.YELLOW_SOFT);
    backBtn = Stylier.backButton();
    backBtn.setVisible(false);
    footer.add(backBtn);
    return footer;
}
```

```
/**
 * Subclasses build their content into {@link #body}.
 */
protected void buildBody() {
}
```

```
protected void showCard(JComponent content) {
    content.setOpaque(true);
    body.add(content, BorderLayout.CENTER);
    revalidate();
    repaint();
}
```

```

protected void goTo(final CoreUI next) {
    App.open(next);
}

protected void linkBack(final CoreUI previous) {
    backBtn.setVisible(true);
    backBtn.addActionListener(e -> goTo(previous));
}
}
'''

```

Core class that inherit other real life object represent classes - CoreUser.java

```

'''
/**
 * User - base class for everyone (sender, reserver, employer).
 * Holds shared personal details with getters/setters.
 */
public class User {
    private int id;
    private String username;
    private String password;
    private String name;
    private String email;
    private String address;
    private String phone;
    private int age;

    public User() {
    }

    public User(String username, String password, String name, String email,
                String address, String phone, int age) {
        this.username = username;

```

```
        this.password = password;
        this.name = name;
        this.email = email;
        this.address = address;
        this.phone = phone;
        this.age = age;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getName() {
        return name;
    }
```

```
}

public void setName(String name) {
    this.name = name;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getPhone() {
    return phone;
}

public void setPhone(String phone) {
    this.phone = phone;
}

public int getAge() {
    return age;
}
```



```

        public void setAge(int age) {
            this.age = age;
        }
    }

    ...

```

Class that holds the Theme (font and colors) - Theme.java

```

...

import java.awt.Color;
import java.awt.Font;

/**
 * Theme - central place for all colors and fonts used across the app.
 * Orange + Yellow themed, modern and soft.
 */
public class Theme {
    // Colors - page backgrounds
    public static final Color BG = new Color(255, 247, 230);
    public static final Color CARD = new Color(255, 255, 252);
    public static final Color CARD_ALT = new Color(255, 238, 205);

    // Orange / yellow accents
    public static final Color ORANGE = new Color(244, 140, 66);
    public static final Color ORANGE_DARK = new Color(218, 110, 40);
    public static final Color ORANGE_SOFT = new Color(255, 221, 175);
    public static final Color YELLOW = new Color(255, 200, 40);
    public static final Color YELLOW_SOFT = new Color(255, 238, 170);

    // Text
    public static final Color TEXT = new Color(74, 58, 42);
    public static final Color MUTED = new Color(150, 126, 104);
    public static final Color WHITE = Color.WHITE;

```

```
public static final Color LINE = new Color(238, 214, 178);
```

```
// Feedback
```

```
public static final Color ERROR = new Color(198, 70, 60);
```

```
public static final Color GOOD = new Color(88, 150, 82);
```

```
// Fonts
```

```
public static final Font TITLE = new Font("Segoe UI", Font.BOLD, 28);
```

```
public static final Font H1 = new Font("Segoe UI", Font.BOLD, 22);
```

```
public static final Font H2 = new Font("Segoe UI", Font.BOLD, 16);
```

```
public static final Font LABEL = new Font("Segoe UI", Font.BOLD, 13);
```

```
public static final Font BODY = new Font("Segoe UI", Font.PLAIN, 14);
```

```
public static final Font SMALL = new Font("Segoe UI", Font.PLAIN, 12);
```

```
public static final Font BUTTON = new Font("Segoe UI", Font.BOLD, 14);
```

```
public static final Font FIELD = new Font("Segoe UI", Font.PLAIN, 15);
```

```
}
```

```
...
```

11. Test and QA

Test ID	Scenario	Statuses
TC01	Admin login	Pass
TC02	Driver Login	Pass
TC03	Track using Track ID	Pass
TC04	Sender Login	Pass
TC05	Reserver login	Failed
TC06	Change parcel states	Pass
TC07	Track unknown parcel (show error message)	Pass